TREEMAPPO: FINE-GRAINED CREDIT ASSIGNMENT FOR AGENTIC LLM TRAINING VIA TOKEN-LEVEL TRAJECTORY BRANCHING

Anonymous authors
Paper under double-blind review

ABSTRACT

We present TreeMAPPO, a reference-policy-free framework for fine-grained credit assignment in agentic large language model training. Rather than assigning a single scalar reward to an entire reasoning trajectory, TreeMAPPO constructs a rollout tree, estimates segment-level contribution from mixed-success leaves, and converts these estimates into dense token-level training signals. The method is designed for mathematical reasoning agents whose trajectories may interleave natural-language reasoning, fenced Python tool calls, tool responses, and final boxed answers. Compared with methods such as TEMPO that exploit shared prefixes among sampled trajectories, TreeMAPPO actively creates new continuations through guided token-level branching, concentrating exploration at structurally useful and model-plausible decision points. Compared with TreeRPO-style tree credit assignment based on empirical outcome comparisons, TreeMAPPO uses a probabilistic MAP estimator that estimates signed segment contribution before producing token-level advantages. Experiments use a resource-efficient split pipeline that rolls out from the base policy, generates advantage-weighted training data, fine-tunes LoRA adapters, and validates checkpoint snapshots. Results show consistent but modest validation gains across several model families, stronger gains in some tool-use settings, and mixed transfer to broader held-out tests.

1 INTRODUCTION

Sparse end-of-trajectory supervision makes credit assignment a central bottleneck in reinforcement learning for reasoning agents. When a long rollout contains many reasoning tokens, optional tool calls, and only a terminal correctness judgment, a single trajectory-level reward provides little guidance about which intermediate decisions should be reinforced or suppressed. The difficulty is especially acute in agentic settings, where early errors may be repaired by later reasoning or external tool feedback.

Recent work has begun to exploit shared-prefix and tree structure to densify supervision, including implicit prefix-tree methods, explicit tree-sampling methods, off-policy prefix-aware methods, and fine-grained agentic branching methods Tran et al. (2025); Yang et al. (2025); Li et al. (2025); Huang et al. (2025); Wang et al. (2026). These methods show that branching structure exposes local comparisons that are unavailable to flat rollout groups. However, existing approaches often rely on naturally occurring prefix overlap, fixed segment boundaries, offline teacher-generated trees, or heuristic credit estimators that do not explicitly represent uncertainty.

This paper develops fine-grained credit assignment for mathematical reasoning agents through token-level trajectory branching. The central idea is to turn a small set of full rollouts into a structured tree of alternative continuations, then infer which segments are helpful, harmful, or uncertain from the pattern of correct and incorrect descendants. We call the resulting method TreeMAPPO (Tree-based Maximum-A-Posteriori Policy Optimization).

Our implementation uses a lightweight tool-calling protocol: the model receives a question and an optional Python tool specification, emits reasoning in text, may invoke the tool via fenced Python markdown blocks, observes the tool response in context, and is required to terminate with a fi-

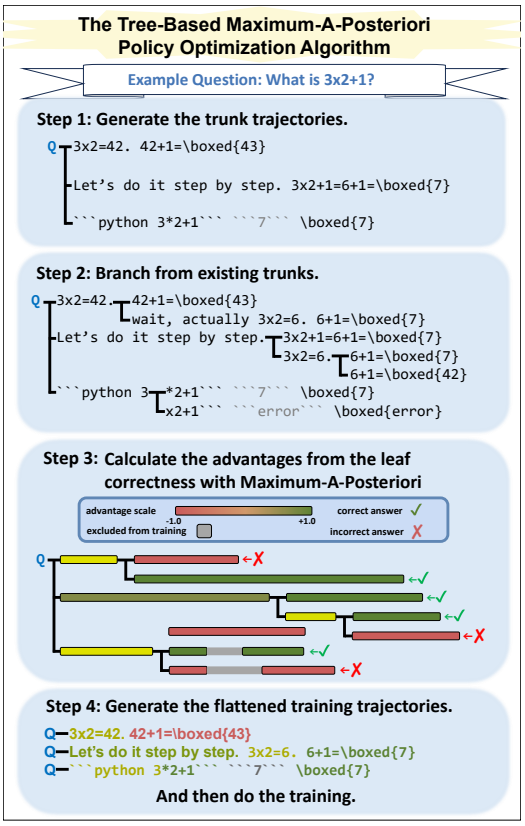

Figure 1: Method overview. TreeMAPPO samples base-policy trajectories, expands selected token-level branch points, infers segment-level contribution and uncertainty from leaf correctness, and converts the resulting posterior estimates into token-level advantages for adapter training.

nal answer wrapped in `\boxed{}`. Rollouts are organized into trees, branch points are chosen at token granularity, and segment-level posterior estimates are transformed into token-level training advantages. The experiments in this submission emphasize a split one-shot pipeline: collect rollouts from the base model, construct the training set once for each condition, train adapters for multiple epochs, and validate the resulting checkpoints. This isolates the proposed credit-assignment mechanism while keeping the compute budget tractable.

Our contributions are:

- We formulate fine-grained credit assignment as posterior inference over signed segment contributions in a trajectory tree.
- We introduce a guided branching policy that prioritizes model-plausible token-level alternatives while discouraging over-fragmentation and over-concentration, with forced first-token branching used to make the intended alternative explicit.
- We describe a practical training pipeline that converts rollout trees into non-redundant training trajectories with token-level advantages.
- We report split-pipeline LoRA experiments showing small positive validation gains in Qwen2.5-7B and Qwen3-4B settings, together with negative results that identify instability at overly aggressive learning rates and mixed held-out transfer.

2 Related Work

Trajectory-level RL for reasoning. Reinforcement learning with verifiable rewards is a standard recipe for mathematical reasoning because final-answer correctness can often be checked automat-

ically. PPO stabilizes policy updates with a clipped surrogate and learned value function Schulman et al. (2017), while GRPO removes the critic by normalizing outcome rewards across sampled responses Shao et al. (2024). Both still face the limitation targeted here: terminal feedback can reinforce or suppress many irrelevant tokens along with the decisive reasoning steps.

Process supervision and verifier-based credit. A complementary line densifies supervision by evaluating intermediate reasoning with human step labels, automatically constructed stepwise supervision, explicit first-error localization, implicit process rewards, or alternative process-reward aggregation rules Lightman et al. (2023); Wang et al. (2024); Hwang et al. (2024); Cui et al. (2025); Cheng et al. (2025). These methods establish the value of process-level signal, but they introduce process evaluators or explicit error-localization/aggregation mechanisms. TreeMAPPO instead infers local contribution from correctness variation across deliberately branched policy trajectories.

Tree, segment, and shared-prefix credit. Structured-rollout methods obtain intermediate credit from shared prefixes, flexible segments, or explicit tree search. TEMPO extracts branch-gated corrections from natural prefix trees Tran et al. (2025); SPO estimates segment-level advantages and includes a tree variant for long traces Guo et al. (2025); TreeRL branches at high-uncertainty intermediate decisions Hou et al. (2025); TreeRPO and TreePO use tree structure for local comparisons or subgroup-relative advantages Yang et al. (2025); Li et al. (2025); and TreeAdv redistributes leaf advantages over entropy-driven rollout forests Cao et al. (2026).

TreeMAPPO is closest to this family, but its differentiating claim is not tree construction alone. It combines model-plausible token-level counterfactual interventions with a joint MAP estimator over latent signed segment contributions and uncertainty, rather than relying on incidental prefix overlap, Monte Carlo segment differences, entropy-only branching, empirical child-group rewards, or direct leaf-advantage redistribution.

Prefix-aware and off-policy tree supervision. Tree-OPO Huang et al. (2025) uses teacher-generated MCTS prefixes and staged advantage estimation to create prefix-aware off-policy training groups. By contrast, TreeMAPPO constructs its trees from behavior-policy rollouts and is reference-policy-free: it does not require a reference policy, offline teacher trees, reward models, or process-level supervision. Our split-pipeline experiments reuse these rollouts across optimization epochs, so the training phase is not fully online or strictly on-policy after the adapter changes.

Fine-grained credit without explicit trees. Tree structure is not the only route to critic-free fine-grained credit: GRPO-λ, OAR, and OPPO localize credit using eligibility-style signals, outcome-sensitivity attribution, or Bayesian value recursion within flat or individually scored trajectories Parthasarathi et al. (2025); Li et al. (2026b;a). TreeMAPPO instead actively generates local counterfactual continuations and infers shared segment contributions from descendant outcomes.

Agentic RL and branching granularity. Long-horizon agents create delayed credit-assignment challenges. GiGPO uses repeated anchor states for local agent-action comparisons Feng et al. (2025), and APPO argues that influential tool-agent decisions are not confined to coarse workflow stages Wang et al. (2026). TreeMAPPO creates local comparisons at token-level reasoning prefixes rather than relying on repeated environment states or procedure-level advantage scaling.

Tool-augmented mathematical reasoning and benchmarks. Our tool setting builds on work showing that language models can interleave reasoning with program execution or external actions Gao et al. (2022); Chen et al. (2022); Yao et al. (2022); Schick et al. (2023). Evaluation uses mathematical reasoning resources including MATH and DeepMath-103K, with additional held-out datasets to test transfer beyond the training distribution Hendrycks et al. (2021); He et al. (2025).

3 Problem Setting and Agent Protocol

We study question-answering agents for mathematical reasoning. Each sample provides a problem statement and a ground-truth final answer used only for post hoc correctness judgment. During rollout, the model may produce plain-text reasoning and, when enabled, invoke a Python executor.

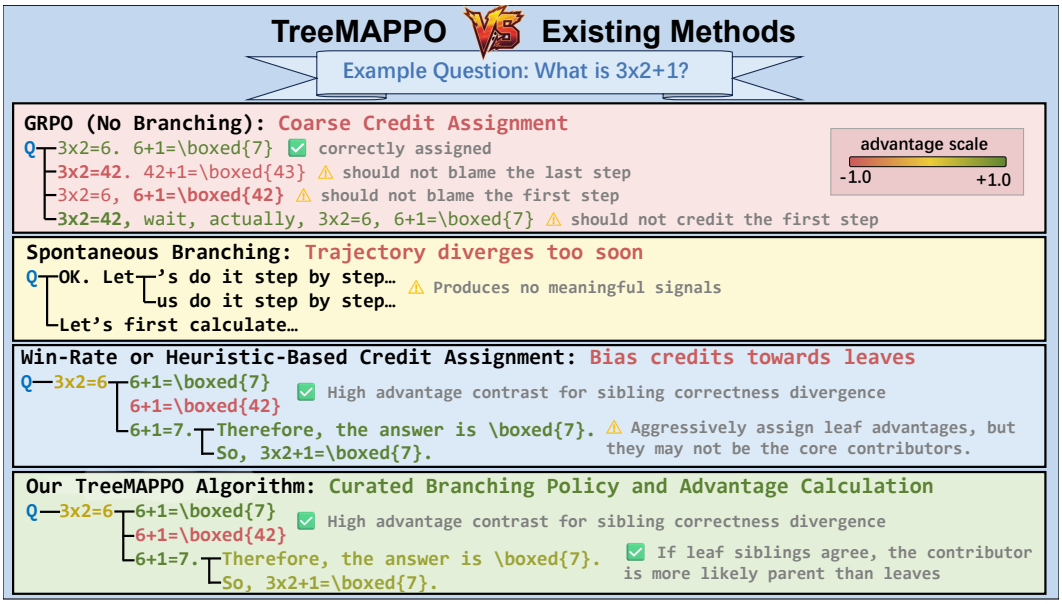

Figure 2: Comparison of credit-assignment structures. Trajectory-level GRPO assigns one normalized outcome signal to every token; TEMPO extracts branch-gated corrections from natural shared prefixes; TreeRPO uses fixed-step tree sampling and empirical child-group rewards; TreeMAPPO combines guided token-level branching with posterior segment-credit inference.

The main methodological question is how to assign credit inside the resulting mixed text–tool trajectory after only a terminal answer judgment is observed. Our experiments keep the prompting and execution protocol fixed across training-data collection and evaluation so that differences across methods are attributable to branching and credit assignment rather than prompt format changes. Appendix A gives the exact tool-calling and rollout protocol.

4 Method

4.1 Overview

For each question, we first sample a set of complete trunk trajectories. We then expand a rollout tree by branching from selected token positions. Each completed leaf trajectory is judged as correct or incorrect using its extracted final boxed answer. The resulting leaf labels provide supervision for segment-level posterior inference. Posterior estimates are transformed into training advantages for model optimization, while branch selection in the split implementation uses rollout structure and model-side token statistics.

4.2 Trajectory Tree Construction

Branching is token-level rather than step-level. A new branch can be created in two ways:

- **Segment split:** split an existing reasoning segment at an internal token position and regenerate from that point.
- **Node expansion:** branch again from an existing branching node, increasing its branching factor.

Each branch preserves the prefix before the selected token position and then creates a new continuation from that point. This construction makes a tree node correspond to an actual intervention in the model's decoded trajectory rather than to a manually annotated reasoning step. Appendix A reports the concrete branch budgets and decoding settings used in experiments.

4.3 MAP Credit Assignment Model

Let ℓ index judged leaves and let i index non-prompt segments. For each leaf ℓ:

- $y_\ell \in \{+1, -1\}$ is the correctness label.
- $x_{\ell,i} \in \{0, 1\}$ indicates whether segment i lies on the path to leaf ℓ.
- Segment i has mean contribution m_i and log standard deviation u_i.

Leaf-level moments are

$$\mu_\ell = \sum_i x_{\ell,i} m_i, \tag{1}$$

$$\tau_\ell = \sqrt{\sum_i x_{\ell,i} \exp(2u_i) + \varepsilon}, \tag{2}$$

$$z_\ell = y_\ell \mu_\ell / \tau_\ell. \tag{3}$$

We use a probit likelihood,

$$p(y_\ell \mid \{m_i, u_i\}) = \Phi(z_\ell), \tag{4}$$

where Φ is the standard normal CDF.

Our implementation initializes prior means at zero and optimizes the negative log posterior

$$\mathcal{J} = -\sum_\ell \log \Phi(z_\ell) + \frac{1}{2\sigma_m^2} \sum_i m_i^2 + \frac{1}{2\sigma_u^2} \sum_i u_i^2. \tag{5}$$

Optimization is performed with gradient steps and backtracking line search, together with a numerically stable implementation of $\log \Phi$ for large negative margins. Concrete prior scales, iteration counts, and numerical clamps are reported in Appendix A.

4.4 Guided Token-Level Branching

The split-pipeline implementation chooses branch points before leaf judgments are available. Branching therefore relies on rollout structure and model-side token statistics rather than posterior uncertainty. For each candidate token position, the branch score multiplies:

1. a structural penalty, and
2. the relative probability of the best alternative token that differs from the already-sampled continuation.

Candidates whose best alternative has relative probability below 0.1 are discarded.

For node expansion, the structural penalty is an exponential branching-factor penalty,

$$\gamma_{\text{node}}(b) = \exp\big(-k_{\text{node}}(b-1)\big), \tag{6}$$

with $k_{\text{node}} = 0.35$ in the reported experiments. For segment splitting, the structural penalty increases with the shorter side of the split relative to average trunk length, discouraging very short fragments.

After a branch point and alternative token are selected, the current guided-branching implementation forces that selected token to be the first token of the new branch continuation. This makes the token-level branch explicit rather than merely restarting generation from the shared prefix and hoping that sampling chooses the intended alternative. We keep the non-forced version as an ablation, but use the forced-token version as the default because it better matches the intended token-level intervention.

4.5 From Posterior Scores to Training Advantages

After tree construction, each segment receives an unnormalized signed score proportional to $m_i / \exp(u_i)$. We additionally apply length-aware rescaling to reduce volatility from very short reasoning spans. The generated training samples then attach token-level advantage values aligned with the input sequence.

The downstream training code consumes aligned token sequences, supervised target tokens, attention masks, token-level advantages, and old token log probabilities. The loss is a clipped policy-ratio surrogate over supervised positions,

$$\mathcal{L}_{\text{train}} = -\frac{1}{|\mathcal{S}|} \sum_{t \in \mathcal{S}} \min \left(\rho_t \tilde{A}_t, \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon) \tilde{A}_t \right), \tag{7}$$

where $\mathcal{S}$ denotes supervised token positions, ρ_t is the ratio between the current token probability and the stored old token probability, and $\tilde{A}_t$ is the token advantage clipped to $[-3, 3]$. This is the implemented optimization interface for injecting segment-level credit into model updates while avoiding the unbounded negative-advantage objective that arises from direct advantage-weighted cross entropy.

4.6 Training-Set Construction

Because many tree leaves share prefixes, naively flattening all leaves would over-train shared segments. We therefore use a greedy de-duplication policy.

1. Score each candidate leaf trajectory by average absolute token advantage.
2. Select the strongest candidate.
3. Mark its segments as consumed.
4. Repeat among trajectories that still contain unconsumed learnable segments.
5. Aggregate selected trajectories across trees and prune the tail with a cumulative average-absolute-advantage cutoff.

Selected trajectories are then emitted under a fixed batching convention so sibling positive and negative samples from the same problem are consumed consecutively; this ordering is an implementation detail rather than an experimental variable.

5 Baselines and Ablations

We compare against the base model and a GRPO-style baseline with multiple complete rollouts but no token-level branching. Ablations independently vary exploration and credit estimation: TEMPO-style branching uses spontaneous shared-prefix divergence; TreeRPO-style credit replaces MAP inference with child-group outcome comparisons; TreeRL-style branching uses entropy-based branch selection; and TreeRL-style credit uses a local–global value-difference estimator. This separates the question of where alternatives come from from the question of how credit is assigned.

6 Experimental Setup

We evaluate TreeMAPPO and matched baselines on Qwen2.5-7B, Qwen3-4B, Gemma-3-4B, Mistral-7B, and Llama-3.1, treating tool-enabled and no-tool settings separately. Training and validation use DeepMath, MATH, and NuminaMath; held-out testing adds AMC2023, GaoKao Math 2024, and CollegeMath. We report pass@1 accuracy as an equal-weight macro average over datasets.

The primary experiments use a split one-shot protocol: rollouts are collected from the base policy, advantage-weighted training data are generated once, and LoRA adapters are trained for multiple epochs before checkpoint validation. This differs from fully online RL, where each policy update would trigger fresh rollouts from the latest checkpoint. The split protocol is less expensive, easier to audit, and better suited to controlled comparisons under limited queue and GPU budgets; it primarily tests the quality of the generated credit-assignment data rather than full closed-loop policy-improvement dynamics.

To reduce compute-budget confounds, GRPO and tree conditions are matched by pre-filter rollout budget. In the main 8-leaf GRPO versus 16-leaf tree comparison, GRPO receives twice as many raw questions per epoch so both settings target the same number of generated leaf trajectories before correctness filtering. Appendix A gives dataset construction, chunk sizes, timing audits, hyperparameters, and validation cadence.

Model and condition	Base	GRPO	TreeMAPPO	Main observation
Qwen2.5-7B no-tool	**0.6366**	0.6353	0.6304	Validation gains do not transfer.
Qwen2.5-7B + tool	0.5742	**0.5850**	0.5836	Both trained methods improve over base.
Qwen3-4B no-tool	0.6940	**0.6999**	0.6997	Both methods improve modestly.
Qwen3-4B + tool	0.6266	0.6487	**0.6694**	TreeMAPPO gives the strongest gain.
Gemma-3-4B no-tool	0.5819	0.5802	**0.5906**	TreeMAPPO is positive but small.
Mistral-7B no-tool	0.1636	**0.1674**	0.1567	Mistral remains unstable.
Llama-3.1 no-tool	0.3930	0.4115	**0.4167**	Both methods improve over base.

Table 1: Main held-out test macro accuracies. Full per-dataset accuracies, confidence intervals, selected checkpoints, and evaluation coverage are reported in Appendix D. Bold entries mark the best macro accuracy in each model-condition block.

7 RESULTS

This section reports the completed experiment inventory. All held-out test rows in the main tables use five rollout trials and all six test datasets; held-out validation rows use six trials. We emphasize conservative conclusions: the evidence supports modest and repeatable validation improvements, but the broader held-out tests show that these gains do not always transfer uniformly.

7.1 VALIDATION ACCURACY TRENDS

Held-out validation shows that both GRPO and TreeMAPPO can extract signal from the split pipeline, although gains are modest and non-monotonic across epochs. Qwen3-4B has the strongest tool-use improvements, while Gemma and Mistral highlight substantial model-family sensitivity. Full validation inventories and the central Qwen2.5-7B no-tool trend are deferred to Appendix B.

7.2 HELD-OUT TESTS

Table 1 reports held-out tests on the broader six-dataset mixture. Each row uses the best checkpoint selected by validation for that method and setting, then evaluates it with five rollout trials when per-trial results are available. The macro average gives equal weight to datasets rather than to individual examples. Checkpoint selection and evaluation coverage are reported separately in Table 5.

The held-out tests sharpen the qualitative picture. In no-tool Qwen2.5-7B, validation gains do not reliably transfer to the broader six-dataset macro average. In tool-enabled Qwen2.5-7B, both trained methods improve over base by about one point, with TreeMAPPO close to GRPO. Qwen3-4B is the strongest positive setting: both methods improve in no-tool testing, and TreeMAPPO substantially outperforms GRPO in the tool setting. Gemma and Llama provide additional positive evidence for TreeMAPPO over their base models, while Mistral remains unstable under this training recipe.

7.3 ABLATION STUDY

The main ablation plan separates branching policy from credit estimation. The TEMPO-style condition changes the branch source to spontaneous shared-prefix divergence. The TreeRPO-style condition changes only the segment-credit estimator to child-group win-rate comparison. The two TreeRL-style conditions separately test entropy-guided branching and local–global value-difference credit. Guided-branching experiments use forced selected branch tokens by default; non-forced branching is retained only as a mechanism ablation.

Table 2 shows that most tree-structured variants improve on their own validation baselines, but the strongest validation gain does not automatically yield the strongest broad test result. TEMPO-style branching has the largest validation gain, suggesting that natural prefix diversity already exposes useful alternatives in this no-tool setting. TreeRPO-style credit is close in validation but weaker on the six-dataset test, indicating that coarse child-group outcome comparisons can overfit the validation distribution. The TreeRL-style credit and combined variants have comparatively strong held-out test macro averages, while entropy-only branching is weaker, suggesting that credit estimation is at least as important as branch-location selection. The forced-token mechanism improves validation

Qwen2.5-7B no-tool variant	Best val. epoch	Val. base	Best val.	Val. gain	Test epoch	Test macro
GRPO baseline	30	0.6468±0.0033	**0.6538±0.0021**	+0.0069	30	**0.6353±0.0118**
TreeMAPPO, non-forced token	20	0.6399±0.0026	0.6472±0.0018	+0.0072	20	0.6294±0.0085
TreeMAPPO, forced token	50	0.6436±0.0025	0.6523±0.0016	+0.0087	50	0.6304±0.0081
TEMPO-style branching	70	0.6418±0.0032	0.6515±0.0020	**+0.0097**	70	0.6326±0.0144
TreeRPO-style credit	40	0.6426±0.0027	0.6519±0.0018	+0.0094	40	0.6272±0.0054
TreeRL-style branching	20	0.6402±0.0045	0.6480±0.0041	+0.0078	20	0.6195±0.0069
TreeRL-style credit	60	**0.6474±0.0044**	0.6528±0.0014	+0.0054	60	0.6334±0.0062
TreeRL combined	40	0.6449±0.0032	0.6507±0.0024	+0.0058	40	0.6341±0.0110
Branch budget 8	10	0.6446±0.0040	0.6471±0.0031	+0.0025	10	0.6274±0.0076
Branch budget 32	60	0.6428±0.0026	0.6503±0.0041	+0.0075	60	0.6333±0.0105

Table 2: Qwen2.5-7B no-tool ablations. Validation numbers are held-out macro averages with 95% confidence intervals from six rollout trials; testing uses five trials and six datasets. Gain is the difference between the best validation checkpoint and the same run's epoch-0 validation score. Bold underlined entries mark the largest value in each numeric column.

over the non-forced version, motivating the default intervention design, although this advantage does not consistently transfer to the held-out test suite.

7.4 Stability Diagnostics

Several completed runs are negative results that shape the current claims. They indicate that the method is sensitive to optimizer aggressiveness, adapter configuration, and model family, motivating the conservative settings used in the main tables. Appendix C summarizes the concrete failed and unstable settings.

The branch-budget sensitivity study is deferred to Appendix F: increasing the branch budget from 8 to 32 improves the held-out test macro average slightly, but the effect is not monotonic enough to be a primary claim.

7.5 Tool-Use Case Study

A representative Qwen2.5-7B tool-enabled TreeMAPPO trajectory illustrates the type of error–repair structure that segment-level credit can distinguish. The model first sets up the right equation and calls Python, but writes an incorrect answer before reading the tool result; after the tool returns 9, the later segment repairs the answer to $\boxed{9}$. TreeMAPPO assigns weak negative credit to the flawed pre-tool span and positive credit to the post-tool repair, with segment advantages $-0.114, +0.222, +0.635, +0.635, +0.576$. The complete colored trajectory and prompts are shown in Appendix G.

8 Discussion

The results point to a trade-off among credit granularity, rollout budget, and optimization stability. GRPO is inexpensive and stable because it assigns one group-normalized terminal signal to full responses, but this simplicity can blur late repairs and intermediate mistakes. Tree-based methods spend more inference budget to expose local alternatives and then train on variable segment-level advantages; this helps when the tree contains meaningful correctness contrasts, but can introduce higher-variance updates when branches fragment unimportant regions or the model family is adapter-sensitive.

Tool use appears to make segmented credit more valuable. Tool trajectories can contain error–feedback–repair transitions, where early reasoning is harmful but later tool-informed reasoning is helpful. This is consistent with the case study and the strongest Qwen3 tool result favoring TreeMAPPO, and motivates analyzing tool-enabled settings separately from no-tool reasoning.

The ablation results should therefore be read as a method-characteristic map rather than a single global ranking. Spontaneous or entropy-based branching can work when models naturally produce diverse alternatives, while guided token branching is more useful when plausible but under-sampled decisions matter. Win-rate and local–global backups can be competitive when correctness changes

align with large subtree boundaries, whereas the MAP estimator is better matched to cases where several shared segments jointly explain mixed descendants.

Several implementation choices follow from this interpretation. Branch selection currently relies on structural penalties and alternative-token probability rather than posterior uncertainty, because judgments are produced after tree construction in the split pipeline. The training loss uses a clipped policy-ratio surrogate with token-level advantages, keeping the optimization interface close to standard policy-gradient practice while preserving tree-based credit assignment. The forced-token branch variant is the default because the branch score evaluates a specific alternative token, so forcing that token makes the generated intervention match the scored decision.

9 Limitations and Future Work

TreeMAPPO incurs additional rollout cost and is evaluated primarily in a split one-shot rather than fully online training regime. Gains are modest and transfer unevenly across models and datasets, motivating larger-scale online experiments, stronger optimizer controls, and repeated independent seeds. The MAP estimator also assumes additive segment contributions and relies on final-answer extraction, assumptions that may become restrictive in more complex agentic environments.

10 Conclusion

We presented TreeMAPPO, a trajectory-tree method for fine-grained credit assignment in agentic LLM training. The core idea is to branch reasoning trajectories at token granularity, infer segment-level contribution from mixed-success descendants, and convert those estimates into training signals for subsequent optimization. Completed split-pipeline experiments show that TreeMAPPO can outperform baseline methods under some configurations, especially when the foundation model and tool-calling context create recoverable intermediate mistakes. The results also reveal a trade-off among finer credit assignment, additional rollout budget, and training stability on trajectories with variable advantages. We therefore view TreeMAPPO not as a uniformly dominant replacement for GRPO, but as a worthwhile option to try as training scale and computation budget increase, particularly for agentic settings where tool feedback can turn early mistakes into later corrections.

AI use statement

Generative AI tools were used to assist with code implementation, experiment orchestration, log inspection, result aggregation, and manuscript editing. All reported results, claims, code changes, and paper text were reviewed by the authors, who take responsibility for the final content of this work.

Ethics statement

This work studies mathematical reasoning benchmarks and does not involve human-subject data collection. The main ethical risks are ordinary risks of improving automated reasoning agents, including possible misuse and over-reliance on generated reasoning traces. We mitigate these risks by evaluating only answer correctness on public mathematical datasets and by reporting negative and mixed results alongside positive findings.

Reproducibility statement

The paper reports the model families, rollout protocol, training hyperparameters, validation cadence, and testing protocol used in the experiments. The implementation keeps explicit judging/scoring passes and concise run summaries for validation and testing so that experiment outputs can be audited and regenerated.

References

Lang Cao, Hui Ruan, Yongqian Li, Peng Chao, Wu Ning, Haonan Song, Renhong Chen, and Yitong Li. Tree-structured advantage redistribution for group-based rl, 2026. arXiv preprint.

Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *arXiv preprint arXiv:2211.12588*, 2022. URL https://arxiv.org/abs/2211.12588.

Jie Cheng, Gang Xiong, Ruixi Qiao, Lijun Li, Chao Guo, Junle Wang, Yisheng Lv, and Fei-Yue Wang. Stop summation: Min-form credit assignment is all process reward model needs for reasoning, 2025. NeurIPS 2025.

Ganqu Cui, Lifan Yuan, Zefan Wang, Hanbin Wang, Wendi Li, Bingxiang He, Yuchen Fan, Tianyu Yu, Qixin Xu, Weize Chen, Jiarui Yuan, Huayu Chen, Kaiyan Zhang, Xingtai Lv, Shuo Wang, Yuan Yao, Xu Han, Hao Peng, Yu Cheng, Zhiyuan Liu, Maosong Sun, Bowen Zhou, and Ning Ding. Process reinforcement through implicit rewards, 2025. arXiv preprint.

Lang Feng, Zhenghai Xue, Tingcong Liu, and Bo An. Group-in-group policy optimization for llm agent training, 2025. arXiv preprint.

Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. Pal: Program-aided language models. *arXiv preprint arXiv:2211.10435*, 2022. URL https://arxiv.org/abs/2211.10435.

Yiran Guo, Lijie Xu, Jie Liu, Dan Ye, and Shuang Qiu. Segment policy optimization: Effective segment-level credit assignment in rl for large language models, 2025. NeurIPS 2025.

Zhiwei He, Tian Liang, Jiahao Xu, Qiuzhi Liu, Xingyu Chen, Yue Wang, Linfeng Song, Dian Yu, Zhenwen Liang, Wenxuan Wang, Zhuosheng Zhang, Rui Wang, Zhaopeng Tu, Haitao Mi, and Dong Yu. Deepmath-103k: A large-scale, challenging, decontaminated, and verifiable mathematical dataset for advancing reasoning. *arXiv preprint arXiv:2504.11456*, 2025. URL https://arxiv.org/abs/2504.11456.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *Advances in Neural Information Processing Systems*, 2021. URL https://arxiv.org/abs/2103.03874.

Zhenyu Hou, Ziniu Hu, Yujiang Li, Rui Lu, Jie Tang, and Yuxiao Dong. Treerl: Llm reinforcement learning with on-policy tree search. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, 2025.

Bingning Huang, Tu Nguyen, and Matthieu Zimmer. Tree-opo: Off-policy monte carlo tree-guided advantage optimization for multistep reasoning, 2025. arXiv preprint.

Hyeonbin Hwang, Doyoung Kim, Seungone Kim, Seonghyeon Ye, and Minjoon Seo. Self-explore: Enhancing mathematical reasoning in language models with fine-grained rewards. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024.

Yizhi Li, Qingshui Gu, Zhoufutu Wen, Ziniu Li, Tianshun Xing, Shuyue Guo, Tianyu Zheng, Xin Zhou, Xingwei Qu, Wangchunshu Zhou, Zheng Zhang, Wei Shen, Qian Liu, Chenghua Lin, Jian Yang, Ge Zhang, and Wenhao Huang. Treepo: Bridging the gap of policy optimization and efficacy and inference efficiency with heuristic tree-based modeling, 2025.

Yu Li, Rui Miao, Tian Lan, and Zhengling Qi. Oppo: Bayesian value recursion for token-level credit assignment in llm reasoning, 2026a. arXiv preprint.

Ziheng Li, Liu Kang, Feng Xiao, Luxi Xing, Qingyi Si, Zhuoran Li, Weikang Gong, Deqing Yang, Yanghua Xiao, and Hongcheng Guo. Outcome-grounded advantage reshaping for fine-grained credit assignment in mathematical reasoning, 2026b. arXiv preprint.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. *arXiv preprint arXiv:2305.20050*, 2023. URL https://arxiv.org/abs/2305.20050.

Prasanna Parthasarathi, Mathieu Reymond, Boxing Chen, Yufei Cui, and Sarath Chandar. Grpo-λ: Credit assignment improves llm reasoning, 2025. arXiv preprint.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*, 2023. URL https://arxiv.org/abs/2302.04761.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. URL https://arxiv.org/abs/2402.03300.

Hieu Tran, Zonghai Yao, and Hong Yu. Exploiting tree structure for credit assignment in rl training of llms, 2025. TEMPO.

Peiyi Wang, Lei Li, Zhihong Shao, Runxin Xu, Damai Dai, Yifei Li, Deli Chen, Yu Wu, and Zhifang Sui. Math-shepherd: Verify and reinforce llms step-by-step without human annotations. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.

Xucong Wang, Ziyu Ma, Yong Wang, Yuxiang Ji, Shidong Yang, Guanhua Chen, Pengkun Wang, and Xiangxiang Chu. Appo: Agentic procedural policy optimization, 2026.

Zhicheng Yang, Zhijiang Guo, Yinya Huang, Xiaodan Liang, Yiwei Wang, and Jing Tang. Treerpo: Tree relative policy optimization, 2025.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022. URL https://arxiv.org/abs/2210.03629.

A Implementation and Protocol Details

Tool-calling protocol. Each prompt contains the problem statement and, when tool use is enabled, a specification that Python can be called by emitting a fenced markdown block. The model may interleave reasoning and tool use. When a fenced Python block closes, generation pauses, the tool executes, and the tool response is appended to the context. The model is instructed to present its final answer in \boxed{}. If generation reaches a control handoff without a boxed answer, the framework requests continuation until a boxed answer is produced or rollout termination conditions are hit.

Rollout and tree hyperparameters. In the default configuration for the main tool-using setup, each tree begins with 4 trunk trajectories and expands to 16 total trajectories. Branch-budget ablations use 8 and 32 total trajectories. We keep a fixed decoding temperature within each tree; in the reported configurations, this value is $T = 0.7$ for trunk generation, continuation, and branch expansion.

Rollout-budget balancing. For the main chunked comparison, GRPO uses 8 complete leaf trajectories per question, while the tree setting uses 16 leaf trajectories per question. To approximately equalize rollout work per training epoch, we assign GRPO twice as many raw questions per chunk: 250 questions for GRPO and 125 questions for tree rollouts. Thus both settings target 2,000 generated leaf trajectories per epoch before correctness filtering. Direct run checks confirm this layout for the Qwen2.5 no-tool run: the first GRPO chunk contains question indices 0–249 with 8 leaves per question, and the first tree chunk contains question indices 0–124 with 16 leaves per question, giving 2,000 leaves in both chunks. The first-run timing summaries are also comparable for Qwen2.5: 235 seconds per GRPO no-tool chunk versus 207 seconds per tree no-tool chunk, and 237 seconds per GRPO tool chunk versus 210 seconds per tree tool chunk. After judging and filtering, the number of accepted training trajectories can differ because all-correct and all-incorrect questions are removed, so the fairness claim is about pre-filter rollout budget rather than identical post-filter training-set size.

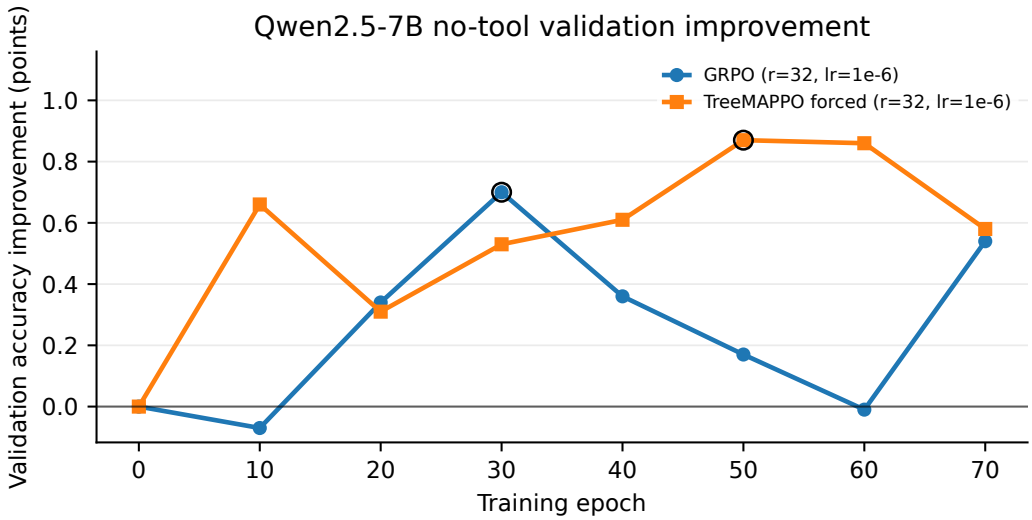

Figure 3: Validation accuracy improvement over the base checkpoint for Qwen2.5-7B in the no-tool setting. Both lines use LoRA rank 32 and learning rate 10^{-6}; the y-axis reports percentage-point change relative to each method's epoch-0 validation accuracy.

MAP-credit optimization details. The hyperparameter configuration uses $\sigma_m = 1.0$, $\sigma_u = 1.0$, $\varepsilon = 10^{-6}$, 120 optimization iterations, and log-standard-deviation clamps in $[-4, 2]$. These settings are fixed across the main TreeMAPPO comparisons unless an ablation explicitly changes the credit-estimation rule.

Datasets. Training and validation are built from DeepMath, MATH, and NuminaMath. The training split contains 5,000 samples per dataset, interleaved across the three in-distribution datasets. The validation split contains 1,000 validation samples per dataset drawn from disjoint index ranges after the training slice. The held-out test split aggregates DeepMath, MATH, NuminaMath, AMC2023, GaoKao Math 2024, and CollegeMath. For datasets with more than 1,000 test rows, 1,000 are sampled with seed 42; for DeepMath, which lacks a dedicated test split in the evaluation pipeline, evaluation rows are drawn from the training split after clearing the reserved training and validation examples.

Training and evaluation protocol. The main stable runs use chunked one-shot training, where each epoch consumes one generated training chunk and recovery restarts at whole-chunk boundaries. Primary experiments use LoRA rank 32 and learning rate 10^{-6}, with Adam state and learning-rate warmup enabled unless stated otherwise. Held-out validation is run at epoch 0 and then every 10 epochs for long runs, with multiple validation rollout trials used to reduce sampling noise. We use LoRA-based fine-tuning for the primary experiments; full-model training is deferred because queueing and memory costs are substantially higher.

B Full Validation Inventory

Table 3 gives the full held-out validation inventory used for checkpoint selection and trend analysis.

C Stability and Negative-Run Diagnostics

Several failed or unhelpful runs influenced the final experiment design. Qwen2.5-7B TreeMAPPO at LoRA rank 32 with learning rate 5×10^{-5} collapsed in earlier validation, while the same rank with learning rate 10^{-6} remained stable. SGD/no-warmup variants did not show useful trends and were removed from the main schedule. Mistral remains difficult under this recipe: its absolute accuracy is low, and TreeMAPPO is below the base model on held-out testing despite a small validation gain.

Model and condition	Method	Trials	Epochs audited	Base	Best epoch	Best	Gain	Last
Qwen2.5-7B no-tool	GRPO	6	0,10,...,70	**0.6468**±0.0033	30	**0.6538**±0.0021	+0.0069	**0.6522**±0.0039
Qwen2.5-7B no-tool	TreeMAPPO	6	0,10,...,70	0.6436±0.0025	50	0.6523±0.0016	**+0.0087**	0.6494±0.0035
Qwen2.5-7B + tool	GRPO	6	0,10,...,70	**0.6142**±0.0033	30	**0.6246**±0.0024	**+0.0104**	0.6171±0.0046
Qwen2.5-7B + tool	TreeMAPPO	6	0,10,...,70	0.6134±0.0034	50	0.6236±0.0033	+0.0101	**0.6181**±0.0016
Qwen3-4B no-tool	GRPO	6	0,10,...,50	**0.6901**±0.0021	50	**0.6991**±0.0038	**+0.0089**	**0.6991**±0.0038
Qwen3-4B no-tool	TreeMAPPO	6	0,10,...,50	0.6894±0.0017	30	0.6944±0.0024	+0.0050	0.6849±0.0047
Qwen3-4B + tool	GRPO	6	0,10,...,50	**0.6446**±0.0030	40	**0.6617**±0.0046	**+0.0172**	0.6521±0.0044
Qwen3-4B + tool	TreeMAPPO	6	0,10,...,50	0.6434±0.0034	50	0.6605±0.0023	+0.0171	**0.6605**±0.0023
Gemma-3-4B no-tool	GRPO	6	0,10,...,50	**0.5640**±0.0022	50	**0.5647**±0.0032	+0.0007	**0.5647**±0.0032
Gemma-3-4B no-tool	TreeMAPPO	6	0,10,...,50	0.5621±0.0030	40	0.5643±0.0029	**+0.0023**	0.5594±0.0034
Mistral-7B no-tool	GRPO	6	0,10,...,50	**0.1747**±0.0043	40	0.1802±0.0013	+0.0055	0.1729±0.0021
Mistral-7B no-tool	TreeMAPPO	6	0,10,...,50	0.1747±0.0019	10	**0.1819**±0.0031	**+0.0072**	**0.1788**±0.0023
Llama-3.1 no-tool	GRPO	6	0,10,...,50	0.4415±0.0038	30	**0.4684**±0.0018	**+0.0269**	**0.4609**±0.0064
Llama-3.1 no-tool	TreeMAPPO	6	0,10,...,50	**0.4454**±0.0035	40	0.4589±0.0029	+0.0135	0.4568±0.0053

Table 3: Audited held-out validation results. Values are pass@1 macro averages on the validation mixture, reported as mean with 95% confidence interval over rollout trials. All rows use six validation trials and epochs that are 0 or multiples of 10. Bold underlined entries mark the largest value within each model-condition comparison block.

		In-distribution datasets			Out-of-distribution datasets			
Model and condition	Method	DeepMath	MATH	NuminaMath	AMC 2023	GaoKao 2024	CollegeMath	Macro avg.
Qwen2.5-7B no-tool	Base	0.6034±0.0030	**0.7400**±0.0056	**0.6160**±0.0211	0.5400±0.0480	0.6000±0.0511	**0.7204**±0.0029	**0.6366**±0.0145
Qwen2.5-7B no-tool	GRPO	0.5886±0.0028	0.7338±0.0056	0.5940±0.0133	0.5200±0.0475	**0.6692**±0.0185	0.7062±0.0042	0.6353±0.0118
Qwen2.5-7B no-tool	TreeMAPPO	**0.6050**±0.0059	0.7362±0.0031	0.5880±0.0073	**0.5900**±0.0480	0.5615±0.0185	0.7016±0.0043	0.6304±0.0081
Qwen2.5-7B + tool	Base	0.5806±0.0042	0.6812±0.0042	0.5540±0.0048	0.4650±0.0720	0.5154±0.0511	0.6488±0.0082	0.5742±0.0118
Qwen2.5-7B + tool	GRPO	0.5806±0.0063	0.6988±0.0053	0.5660±0.0100	**0.4750**±0.0410	**0.5385**±0.0413	0.6514±0.0041	**0.5850**±0.0149
Qwen2.5-7B + tool	TreeMAPPO	**0.5840**±0.0060	**0.7036**±0.0046	**0.5780**±0.0243	0.4600±0.0504	0.5231±0.0185	**0.6530**±0.0028	0.5836±0.0116
Qwen3-4B no-tool	Base	0.6716±0.0035	0.8244±0.0032	0.6240±0.0202	0.5800±0.0392	0.7615±0.0282	0.7022±0.0022	0.6940±0.0109
Qwen3-4B no-tool	GRPO	**0.6846**±0.0038	0.8250±0.0069	**0.6340**±0.0078	**0.6250**±0.0219	0.7231±0.0369	**0.7078**±0.0042	**0.6999**±0.0037
Qwen3-4B no-tool	TreeMAPPO	0.6840±0.0092	**0.8368**±0.0047	0.6120±0.0073	0.5850±0.0250	**0.7769**±0.0554	0.7034±0.0022	0.6997±0.0098
Qwen3-4B + tool	Base	0.6078±0.0056	0.7458±0.0044	0.5900±0.0164	0.5850±0.0250	0.5692±0.0440	0.6616±0.0029	0.6266±0.0099
Qwen3-4B + tool	GRPO	0.6288±0.0063	**0.7734**±0.0073	0.5480±0.0144	**0.6650**±0.0250	0.6077±0.0369	0.6692±0.0069	0.6487±0.0084
Qwen3-4B + tool	TreeMAPPO	**0.6340**±0.0095	0.7670±0.0021	**0.6020**±0.0096	0.6500±0.0410	**0.6846**±0.0282	**0.6788**±0.0035	**0.6694**±0.0101
Gemma-3-4B no-tool	Base	0.4780±0.0067	0.7606±0.0043	0.5580±0.0114	0.4800±0.0646	0.5692±0.0282	**0.6456**±0.0049	0.5819±0.0115
Gemma-3-4B no-tool	GRPO	0.4810±0.0059	0.7610±0.0027	**0.5920**±0.0114	0.4300±0.0240	0.5769±0.0238	0.6402±0.0039	0.5802±0.0093
Gemma-3-4B no-tool	TreeMAPPO	**0.4846**±0.0122	**0.7670**±0.0078	0.5720±0.0130	**0.5050**±0.0567	**0.5769**±0.0238	0.6382±0.0047	**0.5906**±0.0117
Mistral-7B no-tool	Base	0.1910±0.0092	0.1474±0.0035	0.1900±0.0124	**0.0650**±0.0332	**0.2154**±0.0452	0.1728±0.0040	0.1636±0.0129
Mistral-7B no-tool	GRPO	**0.2136**±0.0062	0.1528±0.0063	**0.2560**±0.0237	0.0500±0.0155	0.1538±0.0238	**0.1782**±0.0069	**0.1674**±0.0031
Mistral-7B no-tool	TreeMAPPO	0.2014±0.0058	**0.1562**±0.0061	0.1980±0.0180	0.0350±0.0250	0.1769±0.0511	0.1728±0.0032	0.1567±0.0097
Llama-3.1 no-tool	Base	0.3250±0.0065	0.4954±0.0032	0.4040±0.0100	0.2850±0.0332	0.3846±0.0238	0.4642±0.0052	0.3930±0.0036
Llama-3.1 no-tool	GRPO	**0.3666**±0.0111	**0.5168**±0.0071	0.4340±0.0159	0.2750±0.0268	**0.4077**±0.0452	0.4688±0.0051	0.4115±0.0086
Llama-3.1 no-tool	TreeMAPPO	0.3314±0.0045	0.5146±0.0118	**0.4480**±0.0114	**0.3200**±0.0098	**0.4077**±0.0612	**0.4784**±0.0080	**0.4167**±0.0081

Table 4: Held-out test results by dataset. Values are pass@1 accuracies with 95% confidence intervals from per-trial scores. Bold underlined entries mark the largest value within each model-condition comparison block. Table 6 separately compares forced and non-forced TreeMAPPO variants.

These outcomes motivate treating adapter rank, learning rate, optimizer state, and model-family sensitivity as substantive empirical factors rather than incidental implementation details.

D Full Held-Out Test Details

Table 4 reports the per-dataset held-out test results that underlie the compact main-text table.

E Checkpoint and Mechanism Tables

Table 5 records which checkpoints and evaluations support the held-out test table in the main text. Table 6 provides the forced-token mechanism comparison summarized in the main results.

F Branching-Budget Sensitivity

Our experimental plan includes 8-, 16-, and 32-trajectory variants for the Qwen2.5-7B no-tool setting. The held-out tests do not show a simple monotonic scaling law. Branch budget 32 has a higher held-out test macro average than branch budget 8, but the gap is small and remains within the scale of other run-to-run differences.

Model and condition	Method	Selected epoch	Testing trials	Dataset coverage
Qwen2.5-7B no-tool	Base	0	5	6/6
Qwen2.5-7B no-tool	GRPO	30	5	6/6
Qwen2.5-7B no-tool	TreeMAPPO	50	5	6/6
Qwen2.5-7B + tool	Base	0	5	6/6
Qwen2.5-7B + tool	GRPO	30	5	6/6
Qwen2.5-7B + tool	TreeMAPPO	50	5	6/6
Qwen3-4B no-tool	Base	0	5	6/6
Qwen3-4B no-tool	GRPO	50	5	6/6
Qwen3-4B no-tool	TreeMAPPO	30	5	6/6
Qwen3-4B + tool	Base	0	5	6/6
Qwen3-4B + tool	GRPO	40	5	6/6
Qwen3-4B + tool	TreeMAPPO	50	5	6/6
Gemma-3-4B no-tool	Base	0	5	6/6
Gemma-3-4B no-tool	GRPO	50	5	6/6
Gemma-3-4B no-tool	TreeMAPPO	40	5	6/6
Mistral-7B no-tool	Base	0	5	6/6
Mistral-7B no-tool	GRPO	40	5	6/6
Mistral-7B no-tool	TreeMAPPO	20	5	6/6
Llama-3.1 no-tool	Base	0	5	6/6
Llama-3.1 no-tool	GRPO	30	5	6/6
Llama-3.1 no-tool	TreeMAPPO	40	5	6/6

Table 5: Checkpoint-selection and evaluation-coverage table for the held-out tests in Table 1. All listed rows use per-trial, per-dataset results under the unified judging and scoring pipeline.

Condition	Variant	Best val. epoch	Best val.	Test epoch	Test macro
Qwen2.5-7B no-tool	Non-forced token	20	$0.6472_{\pm 0.0018}$	20	$0.6294_{\pm 0.0085}$
Qwen2.5-7B no-tool	Forced token	50	$\mathbf{0.6523_{\pm 0.0016}}$	50	$\mathbf{0.6304_{\pm 0.0081}}$
Qwen2.5-7B + tool	Non-forced token	70	$0.6193_{\pm 0.0047}$	70	$\mathbf{0.5962_{\pm 0.0044}}$
Qwen2.5-7B + tool	Forced token	50	$\mathbf{0.6236_{\pm 0.0033}}$	50	$0.5836_{\pm 0.0116}$

Table 6: Forced-token mechanism ablation for TreeMAPPO. Forced-token branching improves the selected validation checkpoint in both Qwen2.5-7B settings, but the held-out test difference is small and reverses in the tool setting.

G Qualitative Case Study Details

We inspected generated training trajectories for a Qwen2.5-7B tool-enabled TreeMAPPO run and selected a NuminaMath question that also appears in the no-tool ablation results: "A woman is 42 years of age and her daughter is 8 years old. In a certain number of years, the mother will be three times as old as her daughter. How many years will it take for the mother to be three times as old as her daughter?" The correct equation is $42 + x = 3(8 + x)$, giving $x = 9$.

Figure 6 gives the exact prompts seen by the model in the tool and no-tool TreeMAPPO settings, including the Qwen chat-template markers. Figures 5 and 7 show exact raw trajectory excerpts from TreeMAPPO and available ablations. For the main tool/no-tool comparison, we use the same question. For the ablation panels, we intentionally allow different questions because questions whose sampled leaves are uniformly correct or uniformly incorrect produce zero or near-zero advantages under several credit rules; such cases are uninformative for visualizing how each method distributes credit. The displayed ablation questions are listed in each panel.

The colors are used only to visualize token-level training advantages for readers and automated reviewers; they are not part of the model input or output. Prompt tokens and Python tool outputs have ignored labels and are not colored. Each trajectory uses an adaptive muted red–yellow–green interpolation: the largest absolute advantage in that trajectory saturates to dark red or dark green, so color shows within-trajectory credit structure while the printed numbers show cross-trajectory scale. In the tool-enabled TreeMAPPO trajectory, the model first solves the equation with a Python call but

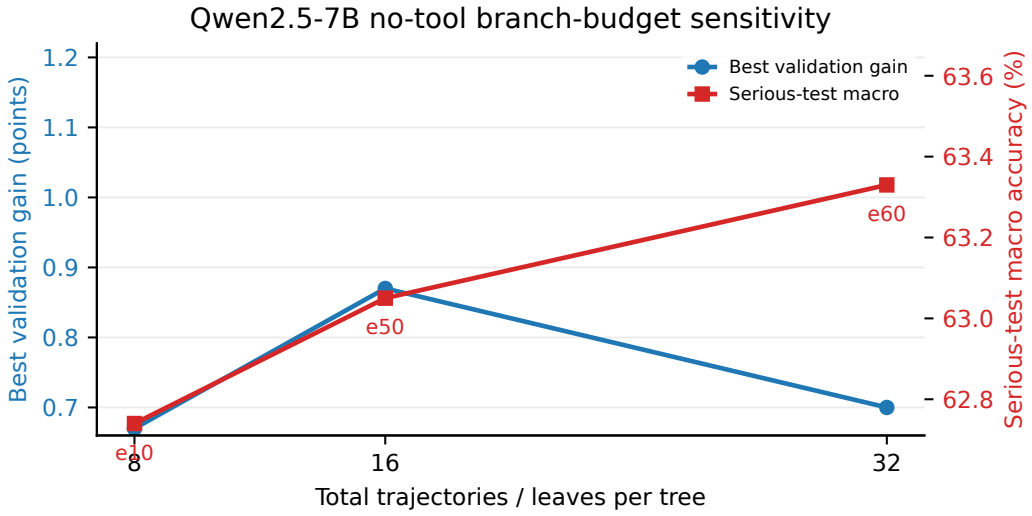

Figure 4: Branching-budget sensitivity for Qwen2.5-7B no-tool TreeMAPPO variants. The validation line reports the best validation improvement over each run's epoch-0 checkpoint; the held-out test line reports the five-trial, six-dataset macro accuracy at the validation-selected checkpoint.

Question. A woman is 42 years old and her daughter is 8 years old. In how many years will the mother be three times as old as her daughter?

Colored raw trajectory excerpt. Let's denote the number of years from now when the mother will be three times as old as her daughter by \(x \). Currently, the mother is 42 years old and the daughter is 8 years old. In \(x \) years, the ages are \(42+x\) and \(8+x\). According to the problem, the mother's age will be three times the daughter's age: \[42+x=3(8+x)\]. Now we solve this equation using Python: ```python from sympy import symbols, Eq, solve ... solution [0] ``` :boxed{6}. So, it will take 6 years for the mother to be three times as old as her daughter.
<tool_response>9</tool_response>
It seems there was an error in my previous calculation. Let's re-solve the equation step by step: \[42+x=3(8+x)\], so \[42+x=24+3x\]. Rearranging terms, \[42-24=3x-x\], \[18=2x\], and \[x=9\]. Therefore, the correct answer is \[\boxed{9}\].

Observed segment advantages. -0.114, $+0.222$, $+0.635$, $+0.635$, $+0.576$.

Figure 5: Qwen2.5-7B tool-use case study. The black tool-response line is ignored by the training loss but explains why the following model segment can repair the answer.

writes an incorrect answer, receives the actual tool response 9, and then corrects the final answer to $\boxed{9}$. The early setup receives weak negative credit, while the post-tool repair receives positive credit.

This appendix is qualitative rather than a substitute for aggregate validation. It illustrates why the TreeMAPPO design combines guided branching with posterior segment credit. GRPO supplies only a whole-response terminal signal, so it cannot distinguish setup, computation, and repair inside a response. TreeRPO- and TreeRL-style backups can create useful sign changes, but their values are coarser because they are backed up from subtree correctness rates rather than inferred as latent segment contributions. TEMPO-style branching and TreeRL-style entropy branching can expose alternative paths, but their branch construction is less tightly coupled to the token-level intervention used by TreeMAPPO. In the tool-enabled TreeMAPPO trajectory, the method preserves the mistaken pre-tool context, observes the tool response, and assigns positive credit to the later repair segment without treating the entire response as uniformly good.

Tool prompt, exact decoded trajectory prefix.

```
<|im_start|>system
You are Qwen, created by Alibaba Cloud.  You
are a helpful assistant.<|im_end|>
<|im_start|>user
You are a helpful agent that solves the
following problem.
Question:  A woman is 42 years of age and
her daughter is 8 years old.  In a certain
number of years, the mother will be three
times as old as her daughter.  How many
years will it take for the mother to be
three times as old as her daughter?
You are encouraged to use the Python code
executor to help you do calculations to
ensure correctness.  You can invoke python
code by emitting a Python markdown code
block like this:
```python
import numpy as np
A = np.array([[2, 1], [1, -1]])
b = np.array([5, 1])
solution = np.linalg.solve(A, b)
solution
```
You should reason step by step, and put the
final answer in a \boxed{}.
Begin your reasoning:<|im_end|>
<|im_start|>assistant
```

No-tool prompt, exact decoded trajectory prefix.

```
<|im_start|>system
You are Qwen, created by Alibaba Cloud.  You
are a helpful assistant.<|im_end|>
<|im_start|>user
You are a helpful agent that solves the
following problem.
Question:  A woman is 42 years of age and
her daughter is 8 years old.  In a certain
number of years, the mother will be three
times as old as her daughter.  How many
years will it take for the mother to be
three times as old as her daughter?
You should reason step by step, and put the
final answer in a \boxed{}.
Begin your reasoning:<|im_end|>
<|im_start|>assistant
```

Figure 6: Exact prompts for the same Qwen2.5-7B case-study question. These are the prompts used for the displayed trajectories; they are not manually rewritten prompts.

TreeMAPPO, no-tool. *Question:* A woman is 42 years old and her daughter is 8 years old. In how many years will the mother be three times as old as her daughter?
The sampled no-tool tree solves the age equation directly, so the posterior has little corrective contrast to exploit.

```
Let's denote the number of years from now when the mother will be three times as old as
her daughter as \(x\). Currently, the mother's age is 42 years and the daughter's age is
8 years. In \(x\) years, the mother's age will be \(42+x\) and the daughter's age will be
\(8+x\). We can set up the equation \[42+x=3(8+x)\], solve it, and obtain \(x=9\). The
final answer is \(\boxed{9}\).
```

Advantages: $+0.074, 0$.

GRPO, no-tool. *Question:* Find the auxiliary equation for the ordinary differential equation with constant coefficients $(x^2D^2 + xD + 1)y = \sin(2\log x)\sin(\log x)$.
GRPO assigns one terminal group-normalized advantage to the entire response.

```
To find the auxiliary equation for the given ordinary differential equation with
constant coefficients, we first recognize that the equation is not in a standard
constant-coefficient form because the operator contains \(x^2D^2+xD+1\). We rewrite
it through the substitution \(t=\log x\), where \(xD\) acts like differentiation with
respect to \(t\). This yields the auxiliary equation \(m^2+1=0\).
```

Advantages: $+1.732$ for this representative response.

TEMPO-style branching. *Question:* Find the auxiliary equation for the ordinary differential equation with constant coefficients $(x^2D^2 + xD + 1)y = \sin(2\log x)\sin(\log x)$.
Spontaneous prefix-tree branching produces a nonzero but fragmented early credit pattern.

```
To find the auxiliary equation for the given ordinary differential equation,
we need to follow these steps. First identify the homogeneous part of the
ODE: \((x^2D^2+xD+1)y=0\). Assume \(y=x^m\), substitute \(y'=mx^{m-1}\) and
\(y''=m(m-1)x^{m-2}\), and simplify to \(m^2+1=0\).
```

Advantages: $+0.162, +0.111, -0.263, +0.217, +0.633$.

TreeRPO-style credit. *Question:* Suppose $g(x) = 5x - 3$. What is $g^{-1}(g^{-1}(14))$?
The child-group win-rate rule gives a strong sign change around the transition from generic inverse derivation to the nested evaluation.

```
To solve for \(g^{-1}(g^{-1}(14))\), first find the inverse of \(g(x)=5x-3\). Set
\(y=5x-3\), solve \(x=(y+3)/5\), and obtain \(g^{-1}(x)=(x+3)/5\). Then compute
\(g^{-1}(14)=17/5\), followed by \(g^{-1}(17/5)=((17/5)+3)/5=32/25\). Therefore, the
final answer is \(\boxed {32/25}\).
```

Advantages: $-0.525, +2.000$.

TreeRL-style credit. *Question:* Given $f(x, y) = e^{-(x+y)}$ for $x > 0, y > 0$, compute $P(Y > X + 1)$.
The local–global value rule creates a coarse negative-to-positive split over the integral setup and evaluation.

```
To compute \(P(Y>X+1)\), integrate \(e^{-(x+y)}\) over \(x>0\) and \(y>x+1\). Set up \(
\int _0^\infty \int _{x+1}^\infty e^{-(x+y)}\,dy\,dx \). The inner integral becomes \(
\int _{2x+1}^\infty e^{-u}\,du=e^{-(2x+1)} \), after which the remaining integral can be
evaluated.
```

Advantages: $-0.219, +0.281$.

TreeRL-style branching. *Question:* Evaluate $\lim_{x\to 0}\left(1/\tan^2 x - 1/x^2\right)$.
Entropy-guided branching yields a long recovery pattern with a negative initial transformation and a positive Taylor-expansion segment.

```
Start by writing \(\tan x=\sin x/\cos x\), so \(1/\tan ^2x=\cos ^2x/\sin ^2x\).
Combining terms gives \((x^2\cos ^2x-\sin ^2x)/(x^2\sin ^2x)\). Then analyze the
numerator and denominator as \(x\to 0\) using Taylor series for \(\sin x\) and \(\cos
x\), which reveals the limiting constant after cancellation.
```

Advantages: $-0.323, +0.805$.

Figure 7: Representative trajectory excerpts for available Qwen2.5-7B no-tool ablations. Unlike the main tool/no-tool comparison, ablation panels may use different questions so that the visualized examples avoid the uniform-correctness regime where several advantage rules collapse to zero. The examples are reconstructed from judged trajectories for qualitative analysis.